

Number: P4013R0

Date: 2026-05-08

Audience: [Library Evolution](#)

Target: C++29

Author: [Hana Dusíková](#)

-
- [Motivation](#)
 - [Storing things in a vector](#)
 - [Raw typeless pointer](#)
 - [Managing lifetime with unique_ptr](#)
 - [shared_ptr](#)
 - [templates and std::polymorphic](#)
 - [std::any](#)
 - [Changes](#)
 - [ABI and impact](#)
 - [Implementations](#)
 - [libc++](#)
 - [libstdc++](#)
 - [MS STL](#)
 - [Wording](#)
 - [Feature test macro](#)

Thanks to Alisdair for nerd-sniping me.

P4013R0: constexpr any of all the things

This paper is making `std::any` usable during constant evaluation.

Motivation

Often you just need to pass a value around, but not interact with it, for that in C++ you have multiple options: template (and keep the type information intact), inheritance (and being forced to use intrusive inheritance in your types), passing allocated (`const`) `void *` pointer (and managing its lifetime), alternatively use `std::unique_ptr` or `std::shared_ptr` with custom deleters, or use a polymorphism (since C++26 we also got `std::polymorphic`).

Each of these options has ups and downs: templates have instantiation and huge cost of AST copying (and in growing executable size), inheritance has virtual tables. But turns out all these already are `constexpr` compatible. Standard already provides a facility to pass value based anything `std::any`, and in constant evaluator an allocation costs approx. same as normal values (at least in clang, anything bigger than `int` is an allocation, [even pointer itself can be an allocation](#)). Using `std::any` in code makes it obvious it's anything and makes code readable by not forcing to think who owns the value, or how it is destroyed. Users shouldn't do such tradeoff, just because `std::any` is not marked `constexpr`.

Storing things in a vector

Let's pretend you want to store object collection of unrelated types in [the preferred container](#).

Raw typeless pointer

```
std::vector<void*> things;
things.emplace_back(new int{42});
things.emplace_back(new std::string{"who owns me?"});
// you are on your own managing their lifetime, constant evaluation won't let you m
ess up, but it will be hard
```

Managing lifetime with unique_ptr

```
using any_unique_ptr = std::unique_ptr<void, Deleter>;
std::vector<any_unique_ptr> things;
things.emplace_back(any_unique_ptr{new int{42}, +[](int * ptr) { delete ptr; }});
things.emplace_back(any_unique_ptr{new std::string{"I drag this thing around"}, +
[](std::string * ptr) { delete ptr; }});
// at least you don't need to manage the lifetime on your own
```

shared_ptr

```
std::vector<std::shared_ptr<void>> things;
things.emplace_back(std::static_pointer_cast<void>(std::make_shared_ptr<int>(42)));
things.emplace_back(std::static_pointer_cast<void>(std::make_shared_ptr<std::string>("a helper function will make this nicer")));
// since C++26 this is constexpr
```

templates and std::polymorphic

You must combine these two together :)

```

struct custom_any {
    virtual ~base() = default;
};

template <typename T> struct specific_any: custom_any {
    T value;
    constexpr specific_any(Args && ... args): value{std::forward<Args>(args)...} {
    }
    virtual ~specific_any() = default;
};

template <typename T> specific_any(T) -> specific_any<T>;
// we can't do this, but we can change language to do it:
// template <typename T> custom_any(T) -> specific_any<T>;

using polymorphic_any = std::polymorphic<custom_any>;
std::vector<polymorphic_any> things;
things.emplace_back(polymorphic_any{std::in_place_type<int>, 42});
things.emplace_back(polymorphic_any{std::in_place_type<std::string>, "I wrote my own any yay"});
// you can also write your own `std::any` directly

```

`std::any`

```

std::vector<std::any> things;
things.emplace_back(42);
things.emplace_back("it's almost like javascript :)");
// after this paper, this will work in constexpr too

```

Changes

Every standard library implementation sets a metadata type (a custom vtable) which contains methods how to destroy / copy / move specific type in `std::any`. Selection which type is used is done typically via a series of conditional statement, querying triviality of the type, size of the type. This paper can be implemented trivially by prepending this sequence with a `if constexpr` statement which will pick the allocating behavior for every type.

Fortunately all these changes can happen inside current specification, and we just need to sprinkle `constexpr` keyword in the spec. That's all the change

ABI and impact

This change doesn't break ABI, is fully compatible with existing code. And make `std::any` not being able to leave constant evaluated code, unless the language gets non-transient `constexpr` allocations. And even then this will be compatible, only the `constexpr` variable containing value stored inside `std::any` would have a non-transient allocation inside instead of a short buffer inlined value.

Implementations

libc++

As described in [changes](#) everything is marked `constexpr` and call of `_Handler::__create` is dispatch thru a helper function which uses `if constexpr` and force allocation. All other functionality is using pointer to the handler put there by the `_LargeHandler::__create` function. Which was also changed to use operator `new` instead of libc++ allocating function `__libc_pp_allocate`. Change is visible on my [github](#) and available on compiler explorer under hana's clang.

libstdc++

Similar change as in [libc++](#) but without a wrapping create function. Plus `any_cast` is a bit more brittle in libc++, and compares function pointer, instead of calling the access itself which should do the check if it's compatible type. Code is visible on my [github](#).

MS STL

Microsoft's STL is using extensively `reinterpret_cast` for pointer tagging, to store `constexpr` `type_info *` and type of storage (trivial / small buffer optimization / allocation). To avoid usage of these I used prototype of `pointer_tagging` from [P3125](#) which does same functionality but it works also during constant evaluation. Another thing which needed to change is due the discrepancy between type and storage is to actually look into the tagged pointer, how the payload is storage and use that approach to access it, instead of blindly going in. Code is available on my [github](#).

Wording

22.7.1 General

[any.general]

- 1 Subclause [any] describes components that C++ programs may use to perform operations on objects of a discriminated type.
- 2 [Note 1: The discriminated type can contain values of different types but does not attempt conversion between them, i.e., 5 is held strictly as an `int` and is not implicitly convertible either to `"5"` or to `5.0`. This indifference to interpretation but awareness

of type effectively allows safe, generic containers of single values, with no scope for surprises from ambiguous conversions. — *end note*

22.7.2 Header <any> synopsis

[any.synop]

```
#include <initializer_list>           // see [initializer.list.syn]
#include <typeinfo>                     // see [typeinfo.syn]

namespace std {
    // [any.bad.any.cast], class bad_any_cast
    class bad_any_cast;

    // [any.class], class any
    class any;

    // [any.nonmembers], non-member functions
    constexpr void swap(any& x, any& y) noexcept;

    template<class T, class... Args>
        constexpr any make_any(Args&&... args);
    template<class T, class U, class... Args>
        constexpr any make_any(initializer_list<U> il, Args&&... args);

    template<class T>
        constexpr T any_cast(const any& operand);
    template<class T>
        constexpr T any_cast(any& operand);
    template<class T>
        constexpr T any_cast(any&& operand);

    template<class T>
        constexpr const T* any_cast(const any* operand) noexcept;
    template<class T>
        constexpr T* any_cast(any* operand) noexcept;
}
```

22.7.3 Class bad_any_cast

[any.bad.any.cast]

```
namespace std {
    class bad_any_cast : public bad_cast {
    public:
        // see [exception] for the specification of the special member functions
        constexpr const char* what() const noexcept override;
    };
}
```

¹ Objects of type `bad_any_cast` are thrown by a failed `any_cast`.

```
constexpr const char* what() const noexcept override;
```

² Returns: An implementation-defined NTBS.

22.7.4 Class `any`

[any.class]

22.7.4.1 General

[any.class.general]

```
namespace std {
    class any {
    public:
        // [any.cons], construction and destruction
        constexpr any() noexcept;

        constexpr any(const any& other);
        constexpr any(any&& other) noexcept;

        template<class T>
            constexpr any(T&& value);

        template<class T, class... Args>
            constexpr explicit any(in_place_type_t<T>, Args&&...);
        template<class T, class U, class... Args>
            constexpr explicit any(in_place_type_t<T>, initializer_list<U>,
Args&&...);

        ~any();

        // [any.assign], assignments
        constexpr any& operator=(const any& rhs);
        constexpr any& operator=(any&& rhs) noexcept;

        template<class T>
            constexpr any& operator=(T&& rhs);

        // [any.modifiers], modifiers
        template<class T, class... Args>
            constexpr decay_t<T>& emplace(Args&&...);
        template<class T, class U, class... Args>
            constexpr decay_t<T>& emplace(initializer_list<U>, Args&&...);
        constexpr void reset() noexcept;
        constexpr void swap(any& rhs) noexcept;

        // [any.observers], observers
```

```
constexpr bool has_value() const noexcept;
constexpr const type_info& type() const noexcept;
};
}
```

- 1 An object of class `any` stores an instance of any type that meets the constructor requirements or it has no value, and this is referred to as the *state* of the class `any` object. The stored instance is called the *contained value*. Two states are equivalent if either they both have no value, or they both have a value and the contained values are equivalent.
- 2 The non-member `any_cast` functions provide type-safe access to the contained value.
- 3 Implementations should avoid the use of dynamically allocated memory for a small contained value. However, any such small-object optimization shall only be applied to types `T` for which `is_nothrow_move_constructible_v<T>` is `true`.

[Example 1: A contained value of type `int` could be stored in an internal buffer, not in separately-allocated memory. — end example]

22.7.4.2 Construction and destruction

[any.cons]

```
constexpr any() noexcept;
```

- 1 *Postconditions:* `has_value()` is `false`.

```
constexpr any(const any& other);
```

- 2 *Effects:* If `other.has_value()` is `false`, constructs an object that has no value. Otherwise, equivalent to `any(in_place_type<T>, any_cast<const T&>(other))` where `T` is the type of the contained value.
- 3 *Throws:* Any exceptions arising from calling the selected constructor for the contained value.

```
constexpr any(any&& other) noexcept;
```

- 4 *Effects:* If `other.has_value()` is `false`, constructs an object that has no value. Otherwise, constructs an object of type `any` that contains either the contained value of `other`, or contains an object of the same type constructed from the contained value of `other` considering that contained value as an rvalue.

```
template<class T>
constexpr any(T&& value);
```

- 5 Let `VT` be `decay_t<T>`.
- 6 *Constraints:* `VT` is not the same type as `any`, `VT` is not a specialization of `in_place_type_t`, and `is_copy_constructible_v<VT>` is `true`.

- 7 *Preconditions:* VT meets the *Cpp17CopyConstructible* requirements.
- 8 *Effects:* Constructs an object of type any that contains an object of type VT direct-initialized with `std::forward<T>(value)`.
- 9 *Throws:* Any exception thrown by the selected constructor of VT.

```
template<class T, class... Args>
constexpr explicit any(in_place_type_t<T>, Args&&... args);
```

10 Let VT be `decay_t<T>`.

11 *Constraints:* `is_copy_constructible_v<VT>` is `true` and `is_constructible_v<VT, Args...>` is `true`.

12 *Preconditions:* VT meets the *Cpp17CopyConstructible* requirements.

13 *Effects:* Direct-non-list-initializes the contained value of type VT with `std::forward<Args>(args)....`

14 *Postconditions:* `*this` contains a value of type VT.

15 *Throws:* Any exception thrown by the selected constructor of VT.

```
template<class T, class U, class... Args>
constexpr explicit any(in_place_type_t<T>, initializer_list<U> il,
Args&&... args);
```

16 Let VT be `decay_t<T>`.

17 *Constraints:* `is_copy_constructible_v<VT>` is `true` and `is_constructible_v<VT, initializer_list<U>&, Args...>` is `true`.

18 *Preconditions:* VT meets the *Cpp17CopyConstructible* requirements.

19 *Effects:* Direct-non-list-initializes the contained value of type VT with `il, std::forward<Args>(args)....`

20 *Postconditions:* `*this` contains a value.

21 *Throws:* Any exception thrown by the selected constructor of VT.

```
constexpr ~any();
```

22 *Effects:* As if by `reset()`.

22.7.4.3 Assignment

[any.assign]

```
constexpr any& operator=(const any& rhs);
```

- 1 *Effects:* As if by `any(rhs).swap(*this)`. No effects if an exception is thrown.
- 2 *Returns:* `*this`.
- 3

Throws: Any exceptions arising from the copy constructor for the contained value.

```
constexpr any& operator=(any&& rhs) noexcept;
```

4 *Effects:* As if by `any(std::move(rhs)).swap(*this)`.

5 *Postconditions:* The state of `*this` is equivalent to the original state of `rhs`.

6 *Returns:* `*this`.

```
template<class T>
```

```
constexpr any& operator=(T&& rhs);
```

7 Let VT be `decay_t<T>`.

8 *Constraints:* VT is not the same type as `any` and `is_copy_constructible_v<VT>` is `true`.

9 *Preconditions:* VT meets the *Cpp17CopyConstructible* requirements.

10 *Effects:* Constructs an object `tmp` of type `any` that contains an object of type VT direct-initialized with `std::forward<T>(rhs)`, and `tmp.swap(*this)`. No effects if an exception is thrown.

11 *Returns:* `*this`.

12 *Throws:* Any exception thrown by the selected constructor of VT.

22.7.4.4 Modifiers

[any.modifiers]

```
template<class T, class... Args>
```

```
constexpr decay_t<T>& emplace(Args&&... args);
```

1 Let VT be `decay_t<T>`.

2 *Constraints:* `is_copy_constructible_v<VT>` is `true` and `is_constructible_v<VT, Args...>` is `true`.

3 *Preconditions:* VT meets the *Cpp17CopyConstructible* requirements.

4 *Effects:* Calls `reset()`. Then direct-non-list-initializes the contained value of type VT with `std::forward<Args>(args)...`

5 *Postconditions:* `*this` contains a value.

6 *Returns:* A reference to the new contained value.

7 *Throws:* Any exception thrown by the selected constructor of VT.

8 *Remarks:* If an exception is thrown during the call to VT's constructor, `*this` does not contain a value, and any previously contained value has been destroyed.

```
template<class T, class U, class... Args>
constexpr decay_t<T>& emplace(initializer_list<U> il, Args&&...
args);
```

9 Let VT be decay_t<T>.

10 *Constraints:* is_copy_constructible_v<VT> is true and is_constructible_v<VT, initializer_list<U>&, Args...> is true.

11 *Preconditions:* VT meets the *Cpp17CopyConstructible* requirements.

12 *Effects:* Calls reset(). Then direct-non-list-initializes the contained value of type VT with il, std::forward<Args>(args)....

13 *Postconditions:* *this contains a value.

14 *Returns:* A reference to the new contained value.

15 *Throws:* Any exception thrown by the selected constructor of VT.

16 *Remarks:* If an exception is thrown during the call to VT's constructor, *this does not contain a value, and any previously contained value has been destroyed.

```
constexpr void reset() noexcept;
```

17 *Effects:* If has_value() is true, destroys the contained value.

18 *Postconditions:* has_value() is false.

```
constexpr void swap(any& rhs) noexcept;
```

19 *Effects:* Exchanges the states of *this and rhs.

22.7.4.5 Observers

[any.observers]

```
constexpr bool has_value() const noexcept;
```

1 *Returns:* true if *this contains an object, otherwise false.

```
constexpr const type_info& type() const noexcept;
```

2 *Returns:* typeid(T) if *this has a contained value of type T, otherwise typeid(void).

3 [Note 1: Useful for querying against types known either at compile time or only at runtime. — end note]

22.7.5 Non-member functions

[any.nonmembers]

```
constexpr void swap(any& x, any& y) noexcept;
```

1 *Effects:* Equivalent to x.swap(y).

```
template<class T, class... Args>
constexpr any make_any(Args&&... args);
```

2 *Effects:* Equivalent to: `return any(in_place_type<T>, std::forward<Args>(args)...);`

```
template<class T, class U, class... Args>
constexpr any make_any(initializer_list<U> il, Args&&... args);
```

3 *Effects:* Equivalent to: `return any(in_place_type<T>, il, std::forward<Args>(args)...);`

```
template<class T>
constexpr T any_cast(const any& operand);
template<class T>
constexpr T any_cast(any& operand);
template<class T>
constexpr T any_cast(any&& operand);
```

4 Let U be the type `remove_cvref_t<T>`.

5 *Mandates:* For the first overload, `is_constructible_v<T, const U&>` is `true`. For the second overload, `is_constructible_v<T, U&>` is `true`. For the third overload, `is_constructible_v<T, U>` is `true`.

6 *Returns:* For the first and second overload, `static_cast<T>(*any_cast<U>(&operand))`. For the third overload, `static_cast<T>(std::move(*any_cast<U>(&operand)))`.

7 *Throws:* `bad_any_cast` if `operand.type() != typeid(remove_reference_t<T>)`.

8 [Example 1:

```
any x(5); //x holds int
assert(any_cast<int>(x) == 5); // cast to value
any_cast<int&>(x) = 10; // cast to reference
assert(any_cast<int>(x) == 10);
```

```
x = "Meow"; //x holds const char*
assert(strcmp(any_cast<const char*>(x), "Meow") == 0);
any_cast<const char*&>(x) = "Harry";
assert(strcmp(any_cast<const char*>(x), "Harry") == 0);
```

```
x = string("Meow"); //x holds string
string s, s2("Jane");
s = move(any_cast<string&>(x)); // move from any
assert(s == "Meow");
any_cast<string&>(x) = move(s2); // move to any
assert(any_cast<const string&>(x) == "Jane");
```

```

string cat("Meow");
const any y(cat);                                //const y holds string
assert(any_cast<const string&>(y) == cat);

any_cast<string&>(y);                             //error: cannot any_cast away const
— end example]

```

```

template<class T>
constexpr const T* any_cast(const any* operand) noexcept;
template<class T>
constexpr T* any_cast(any* operand) noexcept;
9      Mandates: is_void_v<T> is false.
10     Returns: If operand != nullptr && operand->type() == typeid(T) is
true, a pointer to the object contained by operand; otherwise, nullptr.
11     [Example 2:
bool is_string(const any& operand) {
    return any_cast<string>(&operand) != nullptr;
}
— end example]

```

Feature test macro

17.3.2 Header <version> synopsis [version.syn]

```
#define __cpp_lib_any 201606L2026??L // also in <any>
```